

rclone does not encrypt individual files in place. It wraps a storage location (local folder or another remote) with a **crypt** remote. You talk to the crypt remote in plaintext; rclone encrypts names and contents on the way in and decrypts on the way out. The wrapped location only ever sees ciphertext.

Use a **dedicated subdirectory** on the wrapped remote. Mixing encrypted and unencrypted data in the same folder is a common source of confusion. File size (within ~16 bytes) and modification time are **not** encrypted. Passwords stored in `rclone.conf` are only lightly obscured unless you also encrypt the config file. Keep the two passphrases somewhere you cannot lose them; without them the data is gone.

0. Prerequisites

Bash

```
rclone version
rclone obscure 'use-a-long-passphrase-here'
rclone obscure 'use-a-different-salt-here'
```

`rclone obscure` turns a plaintext passphrase into the string rclone expects in config, flags, and env vars. Store the **plaintext** somewhere safe; store the **obscured** form in scripts.

Example values used below (replace them):

Bash

```
PASS='correct horse battery staple'
SALT='different long salt phrase'
ENC_DIR="$HOME/rclone-cipher"    # ciphertext lives here
PLAIN_DIR="$HOME/rclone-plain"   # scratch plaintext
mkdir -p "$ENC_DIR" "$PLAIN_DIR"
echo 'hello from rclone crypt' > "$PLAIN_DIR/note.txt"
mkdir -p "$PLAIN_DIR/docs"
echo 'folder contents' > "$PLAIN_DIR/docs/readme.md"
```

1. Command line only

Three CLI styles, strongest to most disposable.

1a. Non-interactive config (`rclone config create`)

This writes a named remote into `rclone.conf` without the wizard.

Bash

```
# optional: keep this demo out of your real config
export RCLONE_CONFIG="$HOME/.config/rclone/rclone-crypt-demo.conf"

rclone config create secret crypt \
  remote "$ENC_DIR" \
  filename_encryption standard \
  directory_name_encryption true \
  --obscure \
  password "$PASS" \
  password2 "$SALT"
```

`--obscure` tells rclone to obscure the plaintext passwords as it stores them.
`filename_encryption` must be `standard`, `obfuscate`, or `off` — not `true` / `false`.
`directory_name_encryption` is `true` or `false`.

Check it:

Bash

```
rclone config show secret
rclone listremotes
```

`remote` can also be another rclone remote, e.g. `gdrive:Backups/encrypted` or `s3:mybucket/crypt`. Prefer a path inside the remote, not the remote root.

1b. Connection string (no named remote)

Useful for one-off commands. Passwords must already be obscured.

Bash

```
OBSCURED_PASS=$(rclone obscure "$PASS")
OBSCURED_SALT=$(rclone obscure "$SALT")

rclone ls
":crypt,remote='$ENC_DIR',filename_encryption=standard,directory_name_encryption=true,password='$OBSCURED_PASS',password2='$OBSCURED_SALT':"
```

Same idea wrapping a cloud remote:

Bash

```
rclone ls
":crypt,remote='gdrive:Backups/encrypted',password='$OBSCURED_PASS',password2='$OBSCURED_SALT':"
```

1c. Flags on a named or on-the-fly crypt

Bash

```
rclone ls secret: \
  --crypt-remote "$ENC_DIR" \
  --crypt-filename-encryption standard \
  --crypt-directory-name-encryption true \
  --crypt-password "$OBSCURED_PASS" \
  --crypt-password2 "$OBSCURED_SALT"
```

--crypt-password / --crypt-password2 take the **obscured** string.

Use it: folders and files

All of these talk to the crypt remote. Encryption is automatic.

Bash

```
# create folders
rclone mkdir secret:docs
rclone mkdir secret:photos/2026

# copy files and trees
rclone copy "$PLAIN_DIR/note.txt" secret:
rclone copy "$PLAIN_DIR/docs" secret:docs
rclone copy "$PLAIN_DIR" secret:inbox --create-empty-src-dirs

# list decrypted view
rclone ls secret:
rclone lsd secret:
rclone tree secret:

# sync
rclone sync "$PLAIN_DIR" secret:mirror --progress

# download / decrypt
rclone copy secret:note.txt /tmp/
rclone copy secret:docs /tmp/docs-out

# integrity (use this, not rclone check)
rclone cryptcheck "$PLAIN_DIR" secret:mirror
```

Look at the ciphertext side. Names are unreadable when filename encryption is standard and directory encryption is on:

Bash

```
ls -la "$ENC_DIR"
rclone ls "$ENC_DIR"
```

You should see base32-looking names, not note.txt / docs .

Map encrypted ↔ plaintext names:

Bash

```
rclone backend encode secret: note.txt docs/readme.md
rclone backend decode secret: <encrypted-name>
rclone ls secret: --crypt-show-mapping
```

Mount as a decrypted filesystem (FUSE; Linux/macOS; Windows needs WinFSP):

Bash

```
mkdir -p /tmp/secret-mnt
rclone mount secret: /tmp/secret-mnt --vfs-cache-mode writes
# in another terminal: ls /tmp/secret-mnt
# fusermount -u /tmp/secret-mnt # or rclone rc mount/unmount
```

Filename options (pick one and stay consistent)

filename_encryption	directory names	What you get
standard (default)	true (default)	File and folder names encrypted
standard	false	File names encrypted; folder names stay readable
obfuscate	true/false	Weak name scrambling, longer names allowed
off	ignored	Original names + <code>.bin</code> suffix; contents still encrypted

Do not change these after data is written unless you copy through a new crypt remote with the new settings.

2. Same thing with environment variables

Env vars can:

- 1. Define a whole named remote with no config file
- 2. Supply crypt options globally (`RCLONE_CRYPT_*`)
- 3. Unlock an encrypted `rcclone.conf` (`RCLONE_CONFIG_PASS`)

Remote names used in `RCLONE_CONFIG_<NAME>_...` may only contain letters, digits, and `_` . `secret` is fine; `my-crypt` or `test.enc` will not map cleanly. Env-defined remotes are case-insensitive.

2a. Named crypt remote entirely from env vars

Bash

```
OBSURED_PASS=$(rclone obscure "$PASS")
OBSURED_SALT=$(rclone obscure "$SALT")

export RCLONE_CONFIG_SECRET_TYPE=crypt
export RCLONE_CONFIG_SECRET_REMOTE="$ENC_DIR"
export RCLONE_CONFIG_SECRET_FILENAME_ENCRYPTION=standard
export RCLONE_CONFIG_SECRET_DIRECTORY_NAME_ENCRYPTION=true
export RCLONE_CONFIG_SECRET_PASSWORD="$OBSURED_PASS"
export RCLONE_CONFIG_SECRET_PASSWORD2="$OBSURED_SALT"

rclone listremotes          # includes secret:
rclone mkdir secret:docs
rclone copy "$PLAIN_DIR/note.txt" secret:
rclone ls secret:
rclone copy secret:note.txt /tmp/note-from-env.txt
rclone cryptcheck "$PLAIN_DIR/note.txt" secret:note.txt
```

Pattern: RCLONE_CONFIG_ + remote name + _ + option, all uppercase. You must set _TYPE .

Wrapping another remote that is also defined by env:

Bash

```
export RCLONE_CONFIG_MYGDRIVE_TYPE=drive
export RCLONE_CONFIG_MYGDRIVE_CLIENT_ID=...
export RCLONE_CONFIG_MYGDRIVE_CLIENT_SECRET=...
# token etc.

export RCLONE_CONFIG_SECRET_TYPE=crypt
export RCLONE_CONFIG_SECRET_REMOTE='mygdrive:Backups/encrypted'
export RCLONE_CONFIG_SECRET_PASSWORD="$OBSURED_PASS"
export RCLONE_CONFIG_SECRET_PASSWORD2="$OBSURED_SALT"
```

Only **immediate** options of that remote can be set this way. You cannot set the inner drive token via RCLONE_CONFIG_SECRET_CLIENT_ID . Set the inner remote's own RCLONE_CONFIG_MYGDRIVE_* vars, or use backend-wide vars such as RCLONE_DRIVE_....

2b. Backend-wide crypt flags

These apply to crypt backends when you use flags / connection strings, not as a substitute for a named remote's `_PASSWORD` unless you also point at a crypt remote.

Bash

```
export RCLONE_CRYPT_REMOTE="$ENC_DIR"
export RCLONE_CRYPT_FILENAME_ENCRYPTION=standard
export RCLONE_CRYPT_DIRECTORY_NAME_ENCRYPTION=true
export RCLONE_CRYPT_PASSWORD="$OBSCURED_PASS"
export RCLONE_CRYPT_PASSWORD2="$OBSCURED_SALT"
```

Corresponding flags: `--crypt-remote`, `--crypt-filename-encryption`, `--crypt-directory-name-encryption`, `--crypt-password`, `--crypt-password2`.

2c. Connection string + env (good for scripts)

Bash

```
export RCLONE_CRYPT_PASSWORD="$OBSCURED_PASS"
export RCLONE_CRYPT_PASSWORD2="$OBSCURED_SALT"

rclone copy "$PLAIN_DIR" \

":crypt,remote='$ENC_DIR',filename_encryption=standard,directory_name_encryption=true,password='$RCLONE_CRYPT_PASSWORD',password2='$RCLONE_CRYPT_PASSWORD2':inbox"
```

Avoid putting plaintext passphrases on the command line; process lists and shell history will see them. Obscured values are still secret — treat env and argv as sensitive.

2d. One-shot shell wrapper

Bash

```
#!/usr/bin/env bash
set -euo pipefail

ENC_DIR="${ENC_DIR:-$HOME/rc1one-cipher}"
PASS="${RCLONE_PLAIN_PASS:?set RCLONE_PLAIN_PASS}"
SALT="${RCLONE_PLAIN_SALT:?set RCLONE_PLAIN_SALT}"

export RCLONE_CONFIG_SECRET_TYPE=crypt
export RCLONE_CONFIG_SECRET_REMOTE="$ENC_DIR"
export RCLONE_CONFIG_SECRET_FILENAME_ENCRYPTION=standard
export RCLONE_CONFIG_SECRET_DIRECTORY_NAME_ENCRYPTION=true
export RCLONE_CONFIG_SECRET_PASSWORD="$(rclone obscure "$PASS")"
export RCLONE_CONFIG_SECRET_PASSWORD2="$(rclone obscure "$SALT")"

exec rclone "$@"
```

Usage:

Bash

```
export RCLONE_PLAIN_PASS='correct horse battery staple'
export RCLONE_PLAIN_SALT='different long salt phrase'
chmod +x rclone-secret.sh
./rclone-secret.sh copy ~/docs secret:docs
./rclone-secret.sh ls secret:
```

Do not commit that script with secrets. Prefer a password manager and `RCLONE_PASSWORD_COMMAND` for the **config file** password (next section).

3. Related: encrypt the config file itself

This is separate from crypt remotes. It protects tokens and obscured crypt passwords sitting in `rclone.conf`.

Bash

```
rclone config encryption set          # prompts; writes encrypted
rclone.conf
export RCLONE_CONFIG_PASS='the-config-password'
rclone listremotes                    # no prompt
```

Or:

```
Bash
```

```
export RCLONE_PASSWORD_COMMAND='pass show rclone/config'
```

Losing `RCLONE_CONFIG_PASS` locks you out of the config. It does **not** replace the crypt remote's `password` / `password2` . You need both if you use both.

4. Practical notes

- Same two passphrases + same filename/directory settings = compatible crypt remotes on any machine. Copy ciphertext as-is with `rclone sync wrapped1:enc wrapped2:enc` if both sides already hold encrypted data and the crypt settings match.
- Changing the password means creating a new crypt remote and `rclone copy oldcrypt:newcrypt:` (`rclone` decrypts then re-encrypts).
- `rclone check` is the wrong tool on crypt remotes; use `rclone cryptcheck` .
- standard filename encryption caps names around 143 characters. Use `off` or `obfuscate` only if you understand the tradeoff.
- Env vars and connection strings show up in `ps` and shell history. On a shared host, prefer an encrypted config plus `RCLONE_PASSWORD_COMMAND` , not secrets in `export` .
- For cloud storage, create the ciphertext folder first if you like (`rclone mkdir gdrive:Backups/encrypted`), then point `remote` at that path.

Minimal “I just want encrypted local folders” path:

```
Bash
```

```
mkdir -p ~/cipher
OBSCURED_PASS=$(rclone obscure 'your-passphrase')
OBSCURED_SALT=$(rclone obscure 'your-salt')

rclone copy ~/Documents \

":crypt,remote='$HOME/cipher',password='$OBSCURED_PASS',password2='$OBSCURED_SALT':"

rclone ls \

":crypt,remote='$HOME/cipher',password='$OBSCURED_PASS',password2='$OBSCURED_SALT':"
```

That is the whole mechanism: a crypt overlay, two passphrases, a dedicated ciphertext directory, and either a named remote, a connection string, or `RCLONE_CONFIG_*` environment variables